

TECHNICAL DOCUMENTATION

Narri — Instagram Store E-Commerce Platform

System Design · Database Schema · API Reference · UI Wireframes · SRS



MERN Stack · MongoDB · Express · React · Node.js
Version 1.0 — Prepared for 30-Day Development Sprint
August 2026

Table of Contents	
1. Software Requirements Specification (SRS)	3
2. System Architecture Diagram	5
3. Entity-Relationship (ER) Diagram	6
4. UML Class Diagram	7
5. Complete MongoDB Schema (Mongoose)	8
6. Database Indexes & Optimization	10
7. JWT Authentication Flow Diagram	11
8. Sequence Diagrams (Login, Checkout, Order)	12
9. API Request / Response Reference (JSON)	15
10. UI Wireframes — Storefront	17
11. UI Wireframes — Admin Dashboard	19
12. Responsive Layouts (Desktop / Tablet / Mobile)	20
13. Deployment Architecture	21
14. Developer Documentation & Setup Guide	22
15. Security & Launch Checklist	24

1. Software Requirements Specification (SRS)

1.1 Purpose

This document specifies the functional and non-functional requirements for the Narri e-commerce platform: a customer-facing storefront (targeted at Instagram-driven traffic) and an admin dashboard for the brand owner to manage products, banners, reviews, and orders.

1.2 Scope

The system consists of two applications sharing one backend API: (1) a public storefront for browsing and purchasing products, and (2) a restricted admin dashboard for store management. Out of scope for v1: multi-vendor support, native mobile apps, subscription billing.

1.3 Functional Requirements

ID	Requirement	Module	Priority
FR-01	User can register and log in with email/password	Auth	High
FR-02	User can browse products by category, filter and sort	Catalog	High
FR-03	User can view product detail with images, price, reviews	Catalog	High
FR-04	User can add/remove items in a persistent cart	Cart	High
FR-05	User can checkout and pay via payment gateway	Checkout	High
FR-06	User can view their own order history and status	Orders	Medium
FR-07	User can submit a review for a purchased product	Reviews	Medium
FR-08	Admin can create/update/delete products and stock	Admin	High
FR-09	Admin can upload and manage homepage banners	Admin	High
FR-10	Admin can approve, reject, or delete reviews	Admin	Medium
FR-11	Admin can view orders and update fulfillment status	Admin	High
FR-12	Admin can view a sales summary (revenue, order count)	Admin	Medium
FR-13	System sends order confirmation on successful payment	Notifications	Low

1.4 Non-Functional Requirements

Category	Requirement
Performance	Product listing pages should load within 2 seconds on 4G; images served via CDN (Cloudinary)
Security	Passwords hashed with bcrypt; JWT-based auth; admin routes protected by role middleware; HTTPS only
Usability	Mobile-first design — majority of traffic arrives from Instagram links on mobile devices
Scalability	Stateless API servers behind a single MongoDB Atlas cluster; horizontal scaling possible on Render/Railway
Availability	Target 99% uptime; automated redeploy on push to main branch
Maintainability	Modular route/controller/service structure; environment-based config (no hardcoded secrets)

1.5 User Roles & Permissions

Role	Permissions
Guest	Browse products, view reviews, must register/login to checkout
Customer	All guest permissions + place orders, view own order history, submit reviews
Admin	Full access to admin dashboard — products, banners, reviews, orders, sales data

1.6 Assumptions & Constraints

Single currency (INR/USD, configurable). Single warehouse/no multi-location inventory. Payment gateway (Razorpay/Stripe) handles PCI compliance — card data never touches our own servers. 30-day delivery window assumes solo development with no design handoff delays.

2. System Architecture Diagram

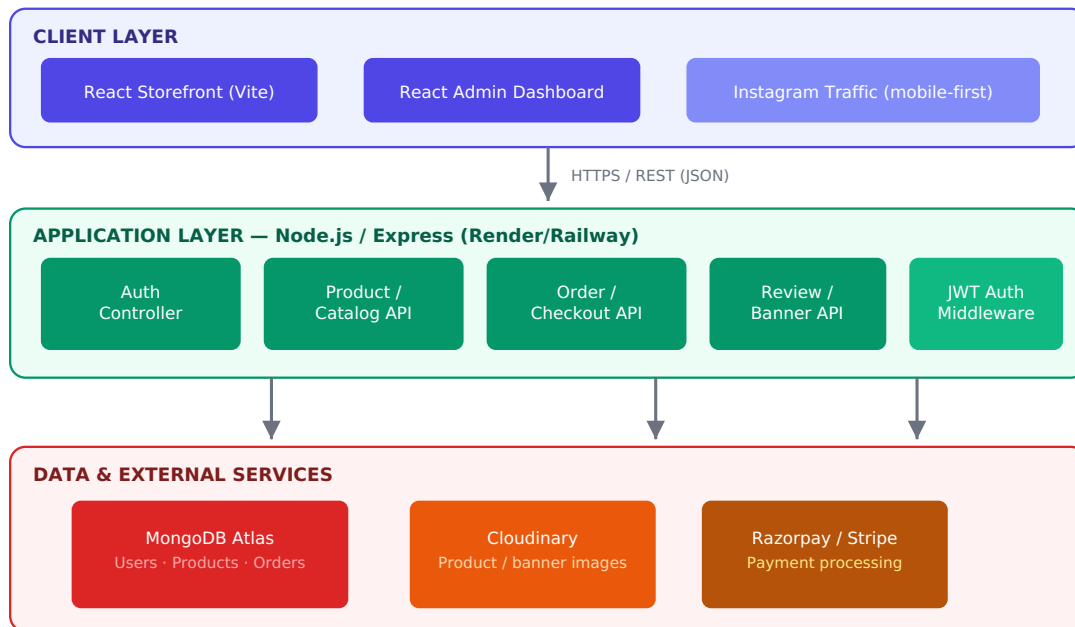


Fig 2.1 — Three-tier architecture: React clients, Express API layer, MongoDB + external services

3. Entity-Relationship (ER) Diagram

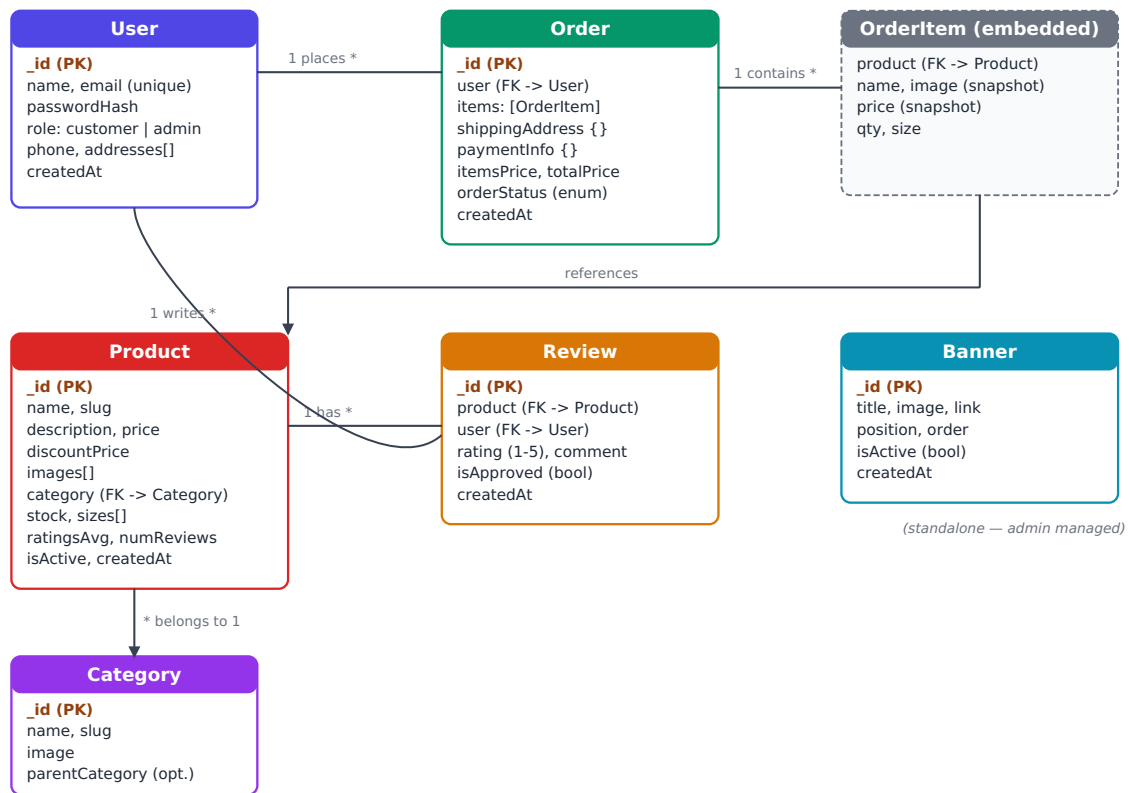


Fig 3.1 — ER diagram with primary/foreign keys and cardinality. Dashed box = embedded sub-document, not a separate collection.

4. UML Class Diagram

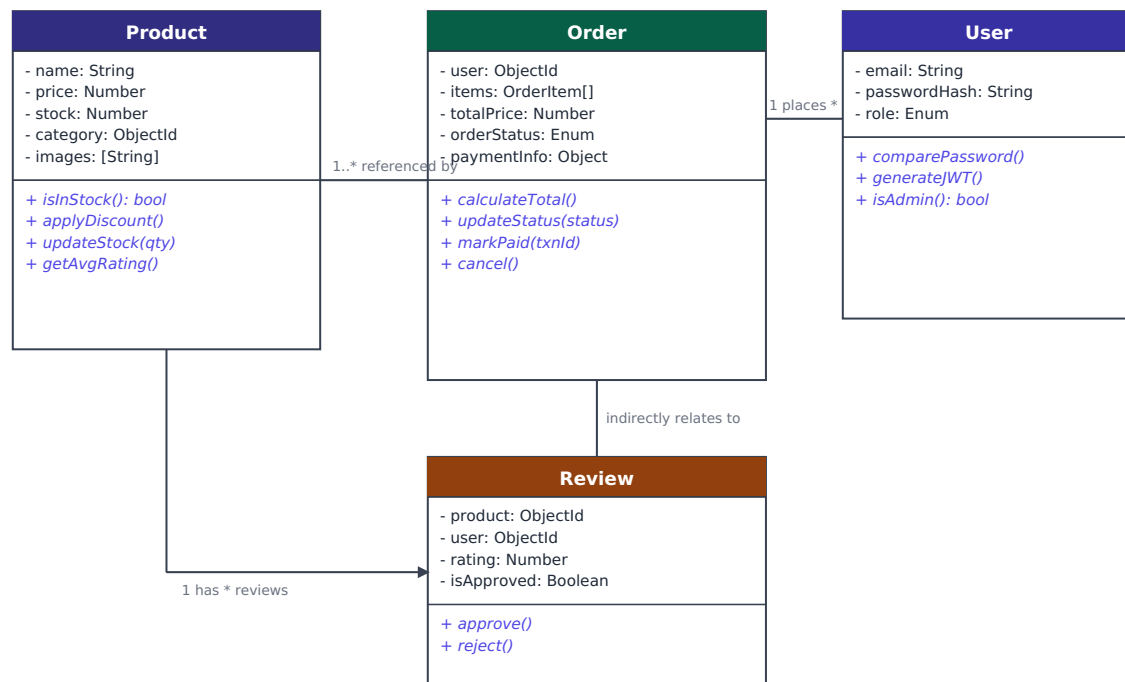


Fig 4.1 — UML class diagram showing key attributes and behaviors (methods represent service-layer logic, not literal Mongoose methods)

5. Complete MongoDB Schema (Mongoose)

5.1 User

```
const userSchema = new Schema({
  name:      { type: String, required: true, trim: true },
  email:     { type: String, required: true, unique: true, lowercase: true },
  passwordHash: { type: String, required: true },
  role:      { type: String, enum: ['customer','admin'], default: 'customer' },
  phone:     { type: String },
  addresses: [{
    label: String, line1: String, line2: String,
    city: String, state: String, pincode: String, isDefault: Boolean
  }],
  isVerified: { type: Boolean, default: false }
}, { timestamps: true });
```

5.2 Category

```
const categorySchema = new Schema({
  name:      { type: String, required: true },
  slug:      { type: String, required: true, unique: true },
  image:     { type: String },
  parentCategory: { type: Schema.Types.ObjectId, ref: 'Category', default: null }
}, { timestamps: true });
```

5.3 Product

```
const productSchema = new Schema({
  name:      { type: String, required: true },
  slug:      { type: String, required: true, unique: true },
  description: { type: String },
  price:     { type: Number, required: true },
  discountPrice: { type: Number },
  images:    [{ url: String, publicId: String }],
  category:  { type: Schema.Types.ObjectId, ref: 'Category', required: true },
  stock:     { type: Number, default: 0 },
  sizes:     [{ label: String, stock: Number }],
  ratingsAverage: { type: Number, default: 0 },
  numReviews: { type: Number, default: 0 },
  isActive:  { type: Boolean, default: true }
}, { timestamps: true });
```

5.4 Order

```
const orderSchema = new Schema({
  user: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  items: [{
    product: { type: Schema.Types.ObjectId, ref: 'Product' },
    name: String, image: String, price: Number, qty: Number, size: String
  }],
  shippingAddress: { line1: String, line2: String, city: String, state: String, pincode: String },
  paymentInfo: {
    method: { type: String, enum: ['razorpay','stripe','cod'] },
    status: { type: String, enum: ['pending','paid','failed'], default: 'pending' },
    transactionId: String
  },
  itemsPrice: Number, shippingPrice: Number, totalPrice: Number,
  orderStatus: { type: String,
    enum: ['placed','processing','shipped','delivered','cancelled'],
    default: 'placed' }
}, { timestamps: true });
```

5.5 Review

```
const reviewSchema = new Schema({
  product:  { type: Schema.Types.ObjectId, ref: 'Product', required: true },
  user:     { type: Schema.Types.ObjectId, ref: 'User', required: true },
  rating:   { type: Number, min: 1, max: 5, required: true },
  comment:  { type: String },
  isApproved: { type: Boolean, default: false }
}, { timestamps: true });
reviewSchema.index({ product: 1, user: 1 }, { unique: true }); // one review per user per product
```

5.6 Banner

```
const bannerSchema = new Schema({
  title:    { type: String },
```

```
image:    { type: String, required: true },
link:     { type: String },
position: { type: String, enum: ['hero', 'mid-page', 'footer'], default: 'hero' },
order:    { type: Number, default: 0 },
isActive: { type: Boolean, default: true }
}, { timestamps: true });
```


6. Database Indexes & Optimization

Collection	Index	Type	Reason
User	{ email: 1 }	Unique	Fast login lookup, prevents duplicate accounts
Product	{ slug: 1 }	Unique	Fast product-detail-page lookup by URL slug
Product	{ category: 1, isActive: 1 }	Compound	Speeds up category listing pages with active-only filter
Product	{ name: "text", description: "text" }	Text	Enables search functionality
Order	{ user: 1, createdAt: -1 }	Compound	Fast "my orders" list, newest first
Order	{ orderStatus: 1 }	Single	Admin dashboard filters by status quickly
Review	{ product: 1, user: 1 }	Unique compound	Enforces one review per user per product
Review	{ product: 1, isApproved: 1 }	Compound	Fast fetch of approved reviews on product page
Banner	{ isActive: 1, order: 1 }	Compound	Fast ordered fetch of live banners for homepage

Optimization notes: Use `.select()` to exclude `passwordHash` from all query responses. Paginate product/order lists with `.limit()` + `.skip()` (or cursor-based pagination once catalog grows past ~5,000 items). Denormalize product name/price/image into `OrderItem` so historical orders remain accurate even if the product is later edited or deleted.

7. JWT Authentication Flow Diagram

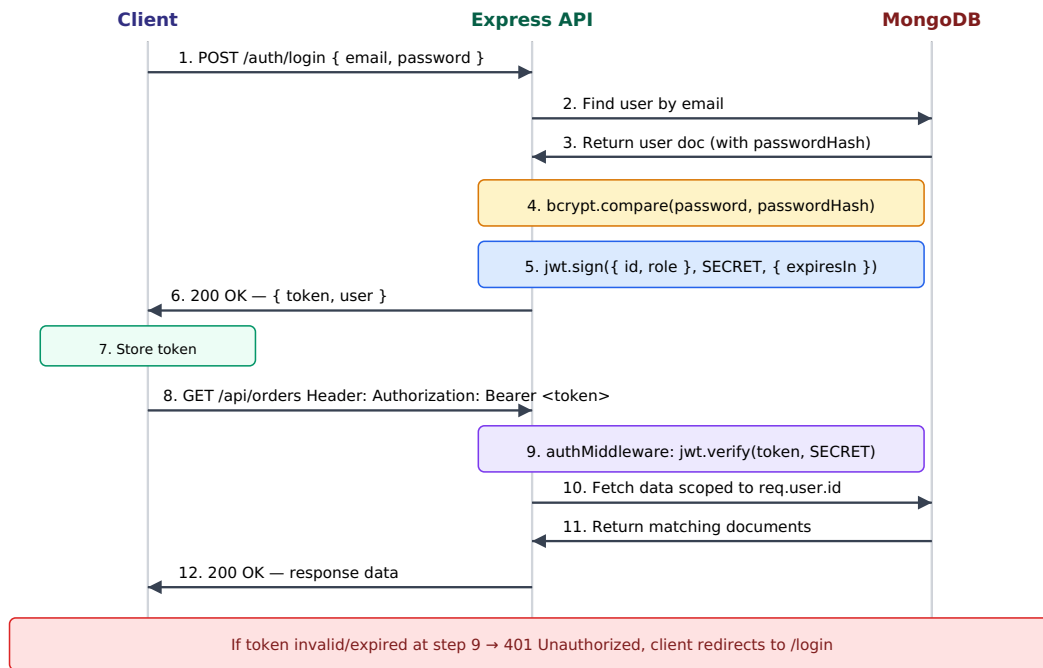
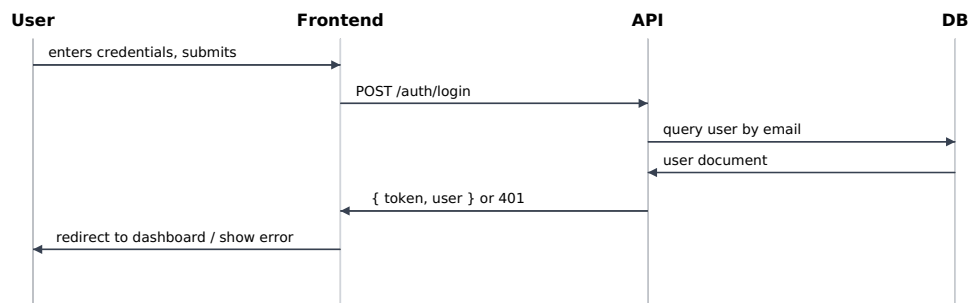


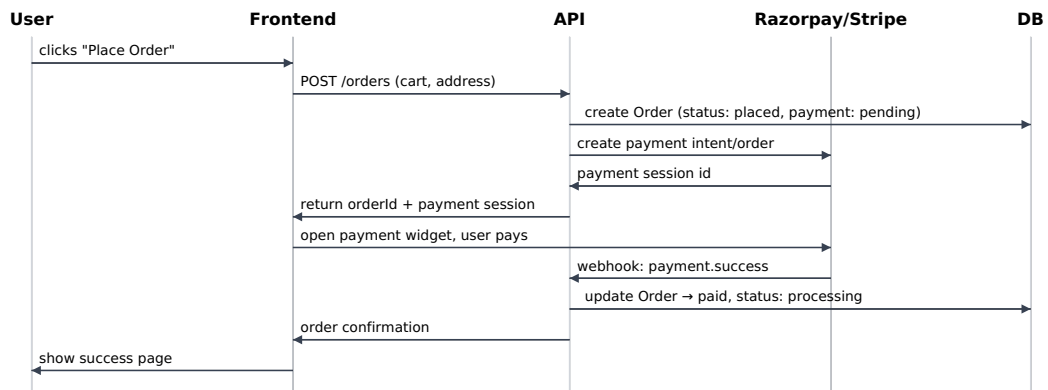
Fig 7.1 — JWT issuance on login and verification middleware on protected routes

8. Sequence Diagrams

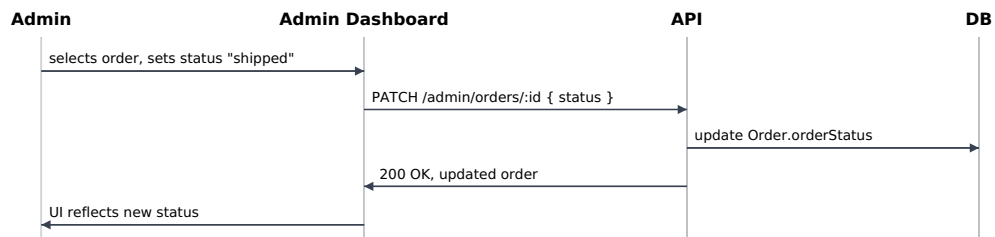
8.1 Login Sequence



8.2 Checkout & Payment Sequence



8.3 Order Fulfillment (Admin) Sequence



9. API Request / Response Reference

9.1 POST /api/auth/register

```
// Request
{
  "name": "Priya Sharma",
  "email": "priya@example.com",
  "password": "*****"
}

// Response 201
{
  "success": true,
  "token": "eyJhbGciOi...",
  "user": { "id": "64f...", "name": "Priya Sharma", "role": "customer" }
}
```

9.2 GET /api/products?category=hoodies&sort=price_asc&page=1

```
// Response 200
{
  "success": true,
  "count": 24,
  "page": 1, "totalPages": 3,
  "products": [
    {
      "id": "66a1...", "name": "Oversized Hoodie — Sand",
      "slug": "oversized-hoodie-sand", "price": 1499,
      "discountPrice": 1199,
      "images": ["https://res.cloudinary.com/.../hoodie1.jpg"],
      "ratingsAverage": 4.5, "numReviews": 18
    }
  ]
}
```

9.3 POST /api/orders (auth required)

```
// Request  Header: Authorization: Bearer <token>
{
  "items": [ { "product": "66a1...", "qty": 2, "size": "L" } ],
  "shippingAddress": {
    "line1": "221B Sector 21", "city": "Faridabad",
    "state": "Haryana", "pincode": "121001"
  },
  "paymentMethod": "razorpay"
}

// Response 201
{
  "success": true,
  "orderId": "66b2...",
  "totalPrice": 2398,
  "razorpayOrderId": "order_Nk3..."
}
```

9.4 PATCH /api/admin/orders/:id (admin only)

```
// Request
{ "orderStatus": "shipped" }

// Response 200
{ "success": true, "order": { "id": "66b2...", "orderStatus": "shipped" } }

// Error — non-admin caller, 403
{ "success": false, "message": "Admin access required" }
```

9.5 POST /api/products/:id/reviews (auth required)

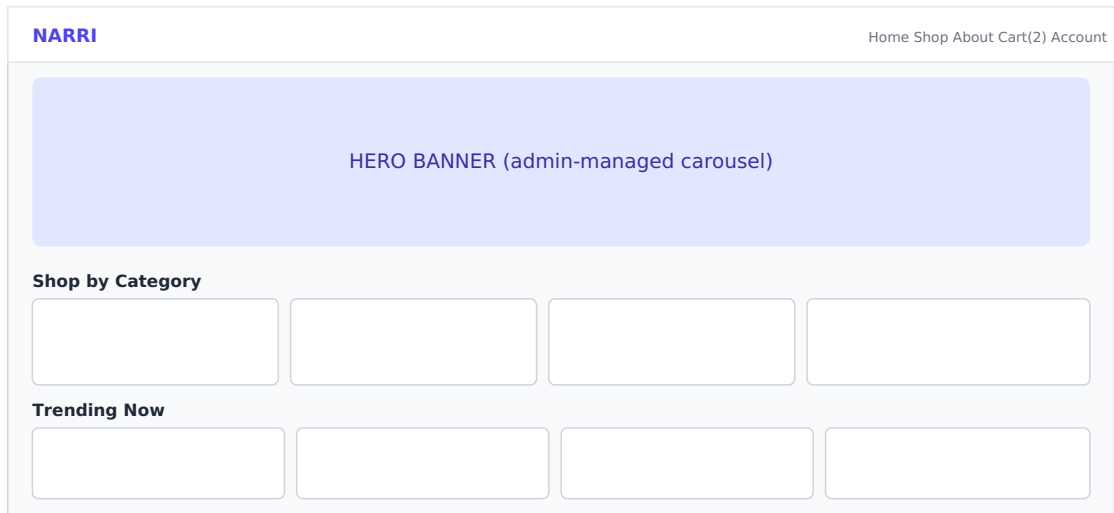
```
// Request
{ "rating": 5, "comment": "Great fit and fast delivery" }

// Response 201
{ "success": true, "review": { "id": "66c9...", "isApproved": false } }

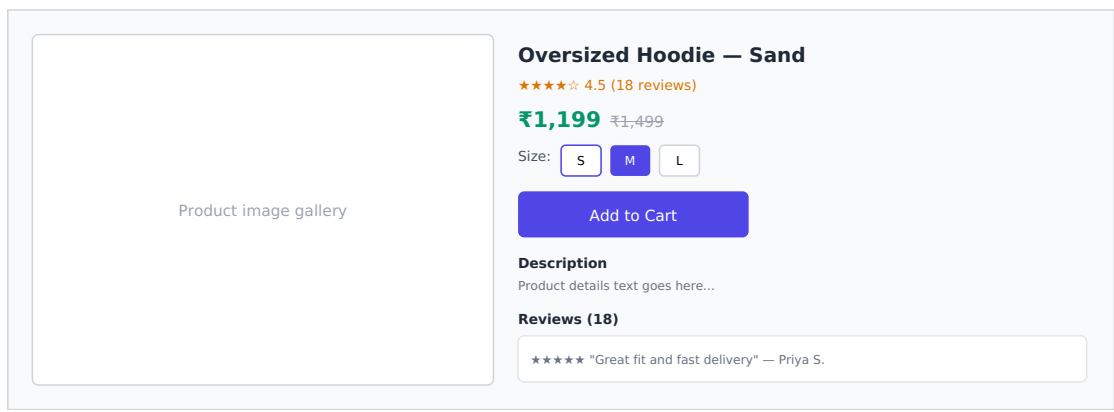
// Review is hidden from product page until admin approves it
```

10. UI Wireframes — Storefront

10.1 Home Page (Desktop)



10.2 Product Detail Page



10.3 Cart & Checkout Page

Your Cart (2 items)

Oversized Hoodie — Sand, Size M
Qty: [-] 1 [+] ₹1,199

Graphic Tee — Black, Size L
Qty: [-] 1 [+] ₹699

Order Summary

Subtotal ₹1,898

Shipping ₹50

Total ₹1,948

Shipping Address

Address form fields...

Proceed to Payment

11. UI Wireframes — Admin Dashboard

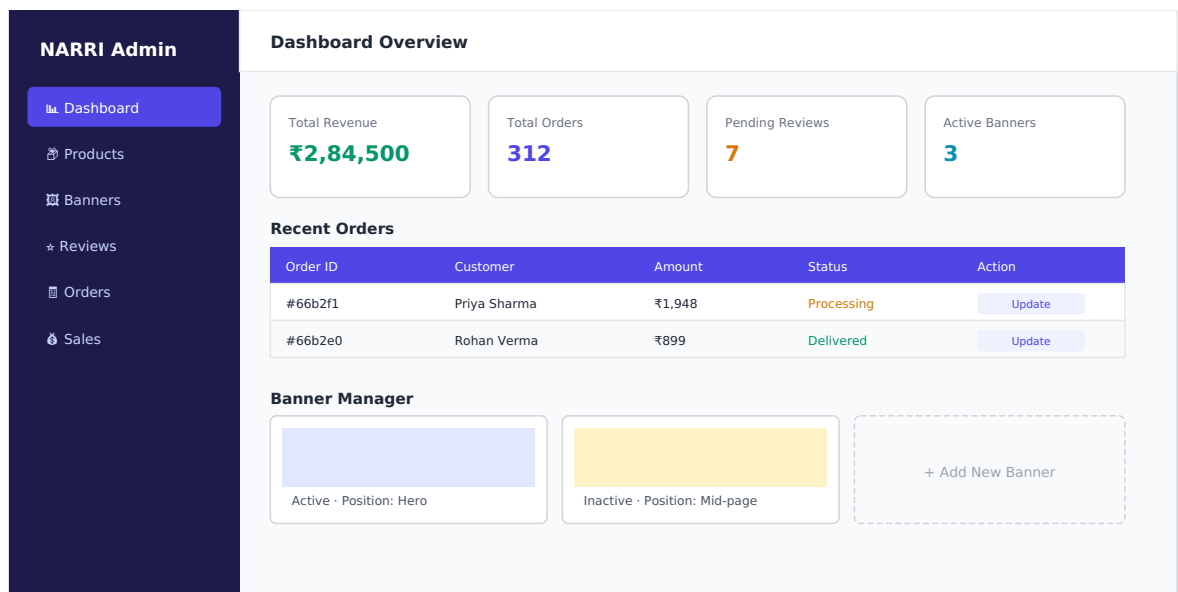
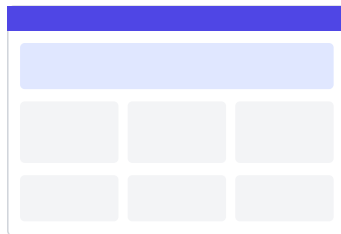


Fig 11.1 — Admin dashboard: stats overview, order management, banner control panel

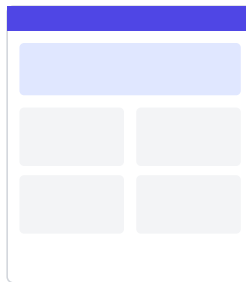
12. Responsive Layouts

Desktop (1280px+)



4-column grid, full nav bar visible

Tablet (768px)



2-column grid,
nav collapses into dropdown

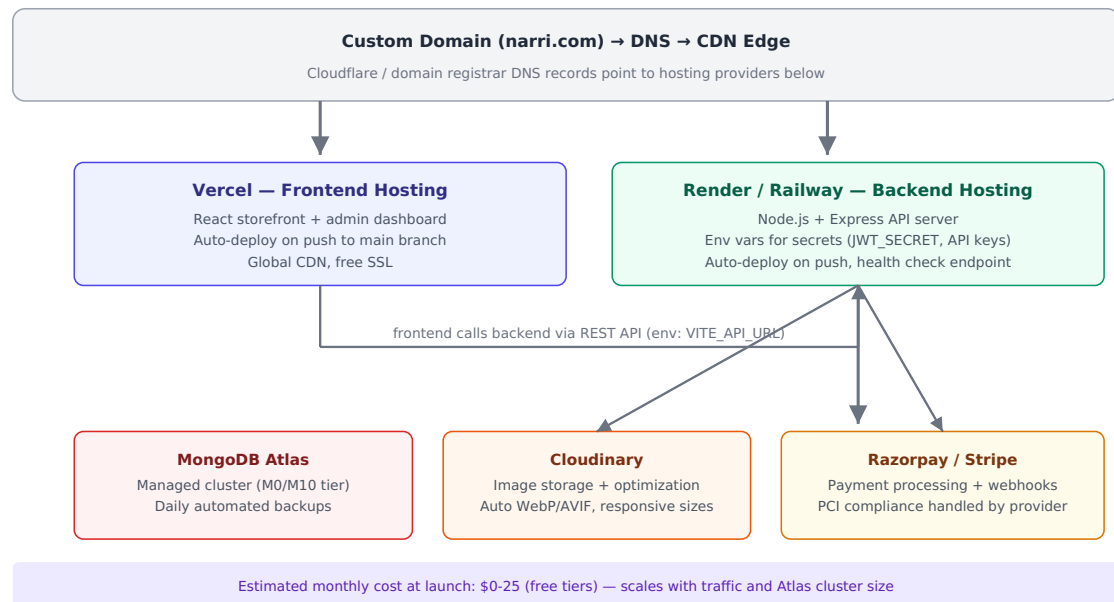
Mobile (375px)



1-column stack,
hamburger menu,
sticky bottom cart bar

Build mobile-first with Tailwind breakpoints: base styles = mobile,
then add sm:/md:/lg: overrides — this matches your Instagram-first traffic pattern

13. Deployment Architecture



14. Developer Documentation & Setup Guide

14.1 Folder Structure

```
narri/
├── client/                                # React storefront
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── context/                    # Cart, Auth context
│   │   ├── api/                       # axios instance + calls
│   │   └── App.jsx
│   └── package.json
├── admin/                                # React admin dashboard
│   ├── src/
│   │   ├── pages/                     # Products, Banners, Orders, Reviews
│   │   └── components/
│   └── package.json
├── server/                               # Express API
│   ├── models/                        # User, Product, Order, Review, Banner
│   ├── controllers/
│   ├── routes/
│   ├── middleware/                    # auth.js, adminOnly.js, errorHandler.js
│   ├── config/                        # db.js, cloudinary.js
│   ├── server.js
│   └── package.json
└── README.md
```

14.2 Environment Variables

Variable	Description
MONGO_URI	MongoDB Atlas connection string
JWT_SECRET	Secret key for signing JWT tokens
JWT_EXPIRES_IN	Token expiry, e.g. 7d
CLOUDINARY_CLOUD_NAME / KEY / SECRET	Image upload credentials
RAZORPAY_KEY_ID / SECRET	Payment gateway credentials
CLIENT_URL	Frontend URL for CORS whitelist
PORT	Backend server port (default 5000)

14.3 Setup Steps

```
# 1. Clone and install
git clone <repo-url> && cd narri
cd server && npm install
cd ../client && npm install
cd ../admin && npm install

# 2. Create .env in /server using the table above

# 3. Run in development (3 terminals)
cd server && npm run dev
cd client && npm run dev
cd admin && npm run dev
```

14.4 Git Workflow

Use a simple trunk-based flow given solo development: commit small, working increments to `main`, tag stable points before major changes (e.g. before adding payments) so you can roll back quickly. Push at the end of each day at minimum — this also keeps Vercel/Render auto-deploys in sync with real progress.

15. Security & Launch Checklist

✓	Item
☐	Passwords hashed with bcrypt (never stored/logged in plaintext)
☐	JWT secret is long, random, and stored only in env vars — never committed
☐	All admin routes protected by role-check middleware, tested with a non-admin token
☐	Input validation on all POST/PATCH routes (e.g. with Joi or express-validator)
☐	Rate limiting on auth routes to slow brute-force attempts
☐	CORS configured to only allow your actual frontend domain(s)
☐	HTTPS enforced everywhere (Vercel/Render provide this by default)
☐	Payment webhook signature verified before trusting "payment success" events
☐	No card/payment data ever touches your own database — gateway handles it
☐	MongoDB Atlas IP allowlist configured (or 0.0.0.0/0 only temporarily during dev)
☐	Error responses don't leak stack traces or internal details in production
☐	Reviews stay hidden until admin-approved, to prevent spam/abuse showing publicly

This document is a planning and reference artifact — treat schema field names and API shapes as a strong starting point, not a rigid spec. Adjust as real requirements surface during the build.